

FEUDALIA - ENTREGA FINAL

Repositorio GitHub de AlgoRitmos

- Enlace: <https://github.com/UCM-FDI-DISIA/proyectoisl-algoritmos>
- Contribuyentes:
 1. Julia Ceregido Andrade (julcereg)
 2. Miriam Elena Cheikho Ferariu (mch-boop)
 3. Andrés García-Redondo Gallego (Andy-uno)
 4. David Morales Bravo (DavidMoralesGit)
 5. Inés Pérez Herrera (inespe10)
 6. Carla Tomás Aguilar (C4rlaa)
 7. Claudia Villodre Pérez (cvillo01)

Índice de contenidos

1. Introducción.....	2
2. Scrum.....	3
2.1. Estructura y funcionamiento del equipo Scrum.....	3
2.2. Historias de Usuario.....	3
2.3. Sprint Reviews.....	4
2.4. Sprint Retrospectives.....	5
2.5. Sprint plannings.....	6
2.6. Product backlog.....	7
2.7. Sprint Backlog.....	7
2.8. Descripción del contenido del repositorio de GitHub.....	9
3. Otras tareas de gestión del proyecto.....	10
3.1 Gestión de riesgos.....	10
3.1.1. Planificación Inicial.....	10
3.1.2. Evolución del Proceso de Gestión de Riesgos:.....	10
3.1.3. Ejemplos de Riesgos Principales y Abordaje.....	12
3.2. Revisiones Técnicas Formales (RTF).....	14
3.2.1. Definición de un Estándar de Revisión.....	14
3.2.2. Implementación de Acciones Correctivas a partir del Feedback.....	15
4. Descripción del trabajo realizado.....	17
5. Código.....	36
5.1. “Test” unitarios.....	37
5.2. Uso de IA generativa.....	37

1. Introducción

El siguiente documento presenta la memoria final del proyecto desarrollado en IS1 por el grupo **AlgoRitmos** y su proyecto, **Feudalia**. El objetivo es recopilar y presentar de forma clara y consensuada por todo el equipo el trabajo realizado a lo largo del cuatrimestre, hablando tanto del resultado final como del proceso seguido.

A lo largo del proyecto se ha aplicado la metodología ágil **Scrum**, organizado en seis sprints, que consisten en reuniones, planificación, retrospectiva, y organización de cada uno, entre otros. Además, se crearon y usaron documentos y metodologías de control de calidad, gestión de riesgos e implementación mediante historias de usuario.

El documento pretende reflejar la evolución completa del equipo, el product backlog, las historias de usuario, y cómo se ha ido transformando el producto final en función de las necesidades puntuales del equipo tanto dentro como fuera de los métodos Scrum.

Se incluye finalmente un análisis de las ramas de desarrollo del proyecto y aportaciones de cada miembro, y el código final para contrastar los avances que se declaren.

2. Scrum

2.1. Estructura y funcionamiento del equipo Scrum

Siguiendo el planteamiento de la clase decidimos trabajar de forma ágil. Aunque Scrum marca roles distintos, decidimos adoptar una organización en la que todos hacemos un poco de todo, tomando decisiones entre todos y ayudándonos siempre que alguien lo necesitara. Esto hizo que el trabajo fuera más fluido y que nadie se quedara bloqueado demasiado tiempo.

Para coordinarnos usamos sobre todo un tablero Kanban en GitHub Projects. Ahí colocamos las historias de usuario y sus tareas siguiendo las columnas típicas: *Ready*, *In Progress*, *Review* y *Done*, añadiendo cuando era necesario una columna de deuda del sprint pasado. Cada tarea tenía información útil (estimación de tamaño, prioridad, tiempo aproximado...) para tener clara la carga de trabajo de cada sprint y repartirnos mejor.

Además del Kanban, nos organizamos con reuniones presenciales periódicas y mensajes por WhatsApp, especialmente para resolver dudas rápidas y mantener a todo el equipo al día.

La wiki también fué una parte clave, ya que ahí guardamos documentos de ayuda, estándares, control de calidad, gestión de errores y referencias que nos servían a todos para trabajar de forma consistente.

A lo largo del proyecto fuimos haciendo tests unitarios para comprobar que las funcionalidades funcionaban como debían y evitar problemas más adelante.

En general, el funcionamiento del equipo fue evolucionando según iban apareciendo errores o tareas nuevas. Como era previsible, no adoptamos del todo todos los métodos ágiles y formalidades en el primer sprint, y fuimos adaptando progresivamente la forma y organización del trabajo a la que avanzamos. Por ejemplo, cuando surgía un problema, lo hablamos entre todos, buscábamos la mejor forma de solucionarlo. En las reuniones de equipo se ponía a todo el mundo al día para aclarar la metodología actual, y el estado actual del proyecto. Así, cualquiera podía ponerse con cualquier parte del proyecto cuando hiciera falta, y eso nos ayudó mucho a mantener el ritmo durante los sprints.

2.2. Historias de Usuario

A lo largo del desarrollo del proyecto se han definido y gestionado diversas historias de usuario, guiando la implementación de cada funcionalidad de Feudalia.

En las primeras etapas del proyecto se redactaron varias historias de usuario iniciales sin una definición clara de usuarios ni una estructura y alineada con la metodología Scrum. Después de recibir una RTF donde se nos indicaba que era confuso tenerlas mezcladas con las correctas posteriores, se decidió eliminarlas ([Ejemplo](#) de “historia” eliminada).

Aunque no se conservan, estas “historias de usuario” iniciales no eran correctas por diversos motivos: no añadían funcionalidad (se centraban en diseño gráfico “atractivo”), los usuarios no eran jugadores (para el Sprint 1 usamos historias centradas en “desarrolladores” y “colaboradores”), algunas eran muy amplias... Poco a poco, identificamos estos errores y diseñamos historias de usuario verdaderas, que pudieran ser útiles en el proceso SCRUM.

Las [historias de usuario finales](#) están todas registradas en la wiki del repositorio, organizadas por el tipo de usuario que las define, visibles en el enlace. Todas siguen la estructura recomendada por Scrum, incluyendo su descripción, criterios de aceptación, [prioridad](#) y [usuario](#) correspondiente.

De las historias de usuario creadas, la gran mayoría se pudieron implementar durante los sprints. Aun así, algunas quedaron pendientes por limitaciones de tiempo que surgieron de una serie de problemas técnicos que requirieron más tiempo del planeado.

Referencias: [Historias de Usuario](#) y [Usuarios de Feudalia](#)

2.3. Sprint Reviews

En esta sección se presentan las Sprint Reviews realizadas a lo largo del desarrollo del proyecto. Las Sprint Reviews son reuniones periódicas que han permitido al equipo mostrar el trabajo completado durante cada sprint, obtener retroalimentación inmediata y ajustar el Product Backlog para los siguientes ciclos. Su objetivo principal ha sido el de fomentar la colaboración, garantizar que el producto evoluciona según los objetivos planteados y facilitar la detección temprana de problemas o desviaciones.

Durante cada Sprint Review, se revisan los incrementos obtenidos, se identifican los avances y las dificultades encontradas, y se proyectan los objetivos del siguiente sprint. Cada miembro del equipo expone su trabajo finalizado y responde preguntas sobre el incremento, asegurando que todos los participantes tengan claridad sobre el estado del proyecto. Además, los resultados de estas revisiones se reflejan en ajustes y refinamientos del Product Backlog, contribuyendo a una planificación más precisa y realista de futuras tareas.

En nuestro proyecto, se realizaron seis Sprint Reviews, cada una documentando detalladamente los avances, problemas encontrados, soluciones implementadas y cambios en el Product Backlog. Los detalles completos de cada revisión, desde la implementación inicial del proyecto hasta la integración del modo multijugador y la versión web, se pueden consultar [en el documento de Sprint Reviews adjunto](#), que describe los incrementos y las retroalimentaciones correspondientes a cada sprint (Sprint 1 a Sprint 6). Este enfoque permite observar cómo ha evolucionado el producto y cómo el equipo ha aprendido y mejorado sus procesos a lo largo del desarrollo.

Referencias: [Revisión de Sprints](#)

2.4. Sprint Retrospectives

Las Sprint Retrospectives nos han permitido como equipo, reflexionar sobre cómo ha transcurrido el último sprint y analizar el rendimiento, la comunicación entre los miembros, los procesos, las herramientas utilizadas, y determinar acciones concretas a mejorar en los siguientes ciclos. El objetivo principal es fomentar la mejora continua, tanto en términos de productividad como de calidad del trabajo, y reforzar la colaboración y comunicación dentro del equipo.

Durante estas retrospectivas, se evaluaron sistemáticamente los aspectos positivos y negativos de cada sprint, proponiendo cambios para optimizar el trabajo. Para llevar a cabo estas retrospectivas, se utilizó el Método Estrella, que clasifica las acciones en las categorías: Start doing (empezar a hacer), Keep doing (seguir haciendo), Stop doing (dejar de hacer), More of (hacer más) y Less of (hacer menos).

A lo largo de los seis sprints, el equipo ha mostrado un progreso notable en varios aspectos:

- Comunicación y coordinación: Desde el Sprint 1 se establecieron reuniones diarias y un desglose organizado de tareas, lo que permitió detectar conflictos y solapamientos de manera temprana. La práctica de intensive commits y la revisión conjunta de tareas ayudó a mantener la transparencia.
- Gestión de riesgos: En los primeros sprints, las dificultades técnicas con Godot y GitHub fueron abordadas de forma colaborativa, fortaleciendo la gestión del equipo y fijando procesos de debugging conjuntos.
- Optimización de procesos: A medida que avanzaron los sprints, se fueron adoptando prácticas como las RTFs, pruebas unitarias tempranas y modularización de códigos para facilitar la implementación de funcionalidades complejas, como el multijugador y la recolección automática.
- Priorización y enfoque: Se aprendió a centrar el esfuerzo en funcionalidades críticas para el producto mínimo viable, reduciendo el tiempo invertido en tareas superficiales o duplicadas y ajustando la planificación a la disponibilidad real del equipo, especialmente durante periodos de alta carga académica.

Las Sprint Retrospectives reflejan la evolución del equipo a la hora de adoptar Scrum y mejorar la productividad: desde un inicio en el que nos centramos en organizar tareas individuales y herramientas, a adoptar un enfoque más colaborativo, estructurado y orientado a la calidad del producto y la buena gestión de los bloqueos. Los detalles completos de cada retrospectiva, incluyendo las decisiones tomadas bajo el Método Estrella para los Sprints 1 a 6, se presentan [a continuación](#), mostrando cómo el equipo ha ido ajustando sus procesos y hábitos de trabajo a lo largo del proyecto.

Referencias: [Retrospectiva Sprints](#)

2.5. Sprint plannings

Las sesiones de Sprint Planning se realizaron al inicio de cada iteración, sirviendo como el eje central para la definición del Sprint Goal y la selección del conjunto de elementos del Product Backlog que constituirían el Sprint Backlog (a ver en los kanban de los sprint backlogs). El propósito principal fue fijar un objetivo de equipo y estimar la carga de trabajo realista para cada sprint.

Para la estimación, utilizamos las técnicas de MoSCoW, clasificación del tamaño en "tallas de camiseta" (T-shirt sizing), y los campos de estimación integrados en nuestros tableros Kanban, lo que facilitó la discusión sobre la complejidad de las funcionalidades.

A medida que ha avanzado el curso y pasado los sprints, hemos notado una evolución en nuestra manera, tanto de estimar el trabajo como de utilizar el contenido de los sprints plannings.

El conocimiento adquirido en las Sprint Retrospectives y Reviews fue integrado sistemáticamente para refinar y optimizar los plannings siguientes. Esta mejora continua se manifiesta en los siguientes puntos clave:

- Ajuste de la Velocidad y Capacidad: Tras los primeros sprints, se aprendió a modular la velocidad del equipo, ajustando el volumen de tareas planificadas para reflejar de forma más precisa la capacidad real, especialmente considerando la carga académica variable del equipo. Esto mitigó el riesgo de comprometer el Sprint Goal.
- Descomposición de las Tareas y Claridad: Los plannings evolucionaron hacia una mayor separación de las Historias de Usuario en tareas. Estas se definieron especificando los criterios de aceptación técnicos y funcionales, lo que redujo la ambigüedad y facilitó tanto la cooperación entre miembros del equipo para resolver problemas, como la detección temprana de dependencias.
- Gestión Proactiva de Bloqueos: La identificación de riesgos técnicos (como las dificultades con la integración de Godot a versión web, o con el funcionamiento del multijugador) en las retrospectivas, llevó a priorizar las tareas más críticas y con mayores dependencias al inicio de los plannings posteriores. Esto garantizó que el equipo abordara los problemas fundamentales antes de centrarse en funcionalidades secundarias.

En resumen, la planificación pasó de una fase inicial de asignación de tareas optimista a un proceso estructurado, impulsado por datos empíricos sobre la velocidad real del equipo y la complejidad técnica intrínseca del proyecto. Esto nos proporcionó una visión común del incremento a conseguir en cada sprint, evitando la pérdida de rumbo durante el desarrollo.

Referencias: [*Sprint Planning*](#)

2.6. Product backlog

La evolución del Product Backlog se llevó a cabo tras cada *Sprint Review*, donde entre otras cosas el equipo revisaba las historias de usuario que debían añadirse, modificarse, reordenarse o cerrarse. Estos cambios quedaron reflejados en los apartados de “*Cambios en el Product Backlog*” documentados tras cada Sprint en el documento de los reviews en la wiki.

Tras los primeros sprints se consiguió un Product Backlog mejor definido y refinado aplicando el criterio MoSCoW para las prioridades, los tamaños, estimaciones de tiempo etc. En los sprint 3 y 4 se añadieron historias más concretas, teniendo en cuenta también sugerencias del profesor (añadir PVE), y las recomendaciones consecuencia de las RTFs. Teniendo como principal objetivo alcanzar el PMV.

En los últimos sprints se ajustó el backlog reordenando prioridades, descartando funcionalidades que no aportaban suficiente valor (inicio de sesión) y añadiendo otras que sí se pretendían añadir para un último refinamiento del proyecto.

Como dicho, los cambios concretos por sprint se encuentran en el siguiente documento: [*Revisión de Sprints*](#)

Referencias: [*Técnicas de Estimación*](#), [*Product Backlog \(Kanban\)*](#), [*Product Backlog \(Wiki\)*](#), [*Revisión de Sprints*](#)

2.7. Sprint Backlog

Como ya ha sido comentado, antes de iniciar cada sprint el equipo se reunía para compartir ideas y llegar a un consenso respecto al enfoque del siguiente sprint. Por lo tanto al proceso de Sprint Planning, siempre le acompañaba inmediatamente la especificación del backlog de cada sprint.

Como equipo decidíamos las historias de usuario en las que pretendíamos trabajar en las siguientes semanas, lo cual se reflejaba en la columna Backlog del kanban de cada sprint. (Estas no serían tareas a ejecutar, luego se dividían en trabajos menos tediosos de implementar). Disponemos de una [*página en la wiki*](#) dedicada a resumir los Backlogs de cada sprint.

Como se comenta en la documentación citada, el sprint 1 careció de un Sprint Backlog, ya que el equipo aún estaba adaptándose a la metodología scrum y realmente no hubo objetivos claros iniciales de trabajo para ese sprint.

En los sprints posteriores fuimos mejorando la organización, creando backlogs por sprint que reflejaran claramente los objetivos del juego que queríamos implementar en las sucesivas semanas. Inicialmente pecamos un poco de optimistas, pero hubo una notable mejora respecto a la estimación del trabajo como ya hemos comentado.

El documento de la wiki ofrece una visión más detallada de lo establecido en cada sprint backlog, puesto que en la columna del kanban solo se encontraban presentes las historias de usuario a solventar. Las tareas de documentación, revisión, deudas de otros sprints, etc. se añadían siempre como issues en To Do.

Principalmente se distinguieron claramente dos tipos de trabajo. Por un lado las tareas orientadas a resolver historias de usuario y aportar nuevos incrementos al producto y por otro las tareas destinadas a solventar deuda técnica, corregir errores o ajustar la planificación, contribuyendo a la estabilidad, corrección y mantenibilidad del proyecto.

Cada Sprint Backlog se definía con el objetivo de alcanzar el Sprint Goal de la planificación. En nuestro caso, dicho objetivo consistía siempre en disponer al final de cada sprint de una versión del juego ejecutable, aunque no necesariamente completa, que incorporase un nuevo incremento funcional.

Para lograrlo, la selección de historias de usuario para el Sprint Backlog resultaba clave, ya que debían aportar funcionalidad real y comprobable al producto, permitiendo validar el progreso del proyecto al finalizar cada iteración. De este modo, el Sprint Backlog no solo servía como herramienta de planificación, sino también como guía para asegurar el avance y el aumento de valor del juego en cada Sprint.

El cumplimiento de este objetivo se reflejaba en las funcionalidades incorporadas al final de cada Sprint, garantizando siempre una versión del juego ejecutable. En los primeros Sprints se añadieron funcionalidades básicas pero fundamentales, como la creación del mapa funcional y el movimiento del personaje, así como la recolección de recursos y su visualización en la interfaz.

Más tarde se amplió la jugabilidad, incorporando mecánicas como la construcción de casas, el reclutamiento de soldados y la gestión de aldeanos, lo que permitió introducir progresivamente elementos estratégicos al juego.

En los últimos Sprints se abordaron funcionalidades de mayor complejidad, como el modo multijugador, los distintos modos de partida, la interacción entre jugadores, así como mejoras estéticas, de experiencia de usuario y de estabilidad general del proyecto. Estas funcionalidades consolidaron el producto como un juego completo y coherente, manteniendo en todo momento la premisa de disponer de un incremento funcional y ejecutable al cierre de cada Sprint, teniendo en cuenta siempre que eso no implicaba que el juego debiera estar completo.

Referencias: [Sprint Backlog](#), [Sprint Kanbans](#)

2.8. Descripción del contenido del repositorio de GitHub

El repositorio de GitHub se ha aprovechado utilizando principalmente dos herramientas para la gestión grupal y seguimiento del desarrollo.

Empezamos cada sprint creando un Project en el formato de un Kanban que usamos como tablero central de organización. Aquí recogemos todas las tareas inicialmente previstas en el sprint (guiándonos por un sprint planning previamente creado), que nos permite tener una visión global del estado actual de trabajo en el sprint y las tareas que deben ser priorizadas.

Las tareas las hemos gestionado con issues, directamente vinculados al Kanban. Cada issue es, o bien una tarea concreta, o una subtarea de otra más compleja que necesita dividirse para mejor trazabilidad. Estos issues los organizamos con el uso de etiquetas de estado (in progress, done...), etiquetas de tamaño (xs, s, m...) , etc, siguiendo la categorización definida previamente en el proceso scrum del equipo.

La segunda herramienta principal es una Wiki ampliamente desarrollada, que usamos como espacio para la documentación del proyecto. Está organizada por las siguientes secciones temáticas:

- Jugar/Descargar: Enlaces directos a las versiones ejecutables del juego.
- Documentación. Esta sección incluye:
 - Documento de Especificación de Requisitos.
 - Documento Técnico.
 - Manual de usuario
- Diseño gráfico: Cuenta con apartados dedicados a la estética, interfaz, diseño, recursos y enlazado de librerías de assets.
- Proceso Scrum. Esta sección incluye:
 - Usuarios de Feudalia con sus historias de usuario correspondientes.
 - Product Backlog.
 - Planificación, revisión y retrospectiva de los sprints.
 - Transcripción de las reuniones.
 - Definición del producto mínimo viable (PMV)
 - Definition of Done (DoD)
- Control de calidad: Incluye tanto los estándares de diseño y calidad propios, como la metodología y estándares para las RTF para el resto de grupos. Existe una copia de cada RTF realizada y nuestro plan de acción tras recibir las RTF.
- Gestión de riesgos: Cuenta con la priorización, supervisión y planificación de los riesgos del proyecto.
- Anexo y ayudas: Documentos de apoyo por y para el equipo.

En conjunto, con esta organización en GitHub hemos podido garantizar una buena organización del trabajo, coherente con la metodología Scrum, a lo largo de todo el proyecto.

3. Otras tareas de gestión del proyecto

Además del trabajo propio de Scrum, durante el proyecto realizamos varias tareas de gestión adicionales que nos ayudaron a mantener el control del desarrollo y la calidad del software.

3.1 Gestión de riesgos

La gestión de riesgos en nuestro proyecto Feudalia siguió un marco metodológico fundamentado en el Modelo de Gestión de Riesgos de Boehm. Este enfoque no solo facilitó la identificación exhaustiva y la priorización temprana de amenazas, sino que estructuró nuestra respuesta a lo largo de todo el ciclo de desarrollo.

Nuestro Plan inicial se documentó formalmente, empleando una tabla de Probabilidad por Impacto (PxI) coherente para clasificar los riesgos. El documento siguió la estructura recomendada de cinco apartados: [Introducción](#), [Priorización](#), [Desarrollo por Riesgo k-ésimo](#), [Planificación Temporal](#) y [Resumen](#).

Para cada riesgo, se incorporaron sistemáticamente estrategias de Reducción, Supervisión y Gestión (RSGR). Este sistema de planificación sentó las bases para que la administración de riesgos evolucionara de un documento estático a un proceso activo de mejora continua, tal como se detalla a continuación.

3.1.1. Planificación Inicial

Nuestro enfoque de partida se centró en el Plan RSGR, el cual identificó y categorizó 30 amenazas potenciales, clasificando 7 de ellas como Intolerables (IN) ([ver tabla detallada en el repositorio](#)). La identificación se organizó por categorías como Arquitectura Técnica, Equipo, Integración y Alcance, permitiendo anticipar desafíos importantes, como la alta complejidad del modo multijugador o los retrasos en tareas clave (que en ocasiones generaron deuda técnica para el Sprint siguiente). El Plan RSGR definió las estrategias de Reducción, Supervisión y Gestión para estas amenazas, sirviendo como un marco de referencia esencial para todas las decisiones de planificación.

3.1.2. Evolución del Proceso de Gestión de Riesgos:

A lo largo del desarrollo, la administración de riesgos evolucionó significativamente, pasando del Plan original a un ciclo de mejora continua impulsado por las dinámicas del equipo y la metodología Scrum.

Inicialmente, el equipo se enfrentó a riesgos relacionados con la colaboración interna no abordados completamente por el plan formal. Al asignar una tarea, se observaron dos problemas:

1. El miembro asignado sentía una alta responsabilidad individual, lo que resultaba en que invirtiera demasiado tiempo tratando de resolver bloqueos técnicos por su cuenta, sin importar el impacto en el lead time del sprint.
2. Otros miembros evitaban colaborar en tareas ya asignadas para ‘no pisar’ o ‘invadir’ el trabajo del compañero, lo que perpetuaba los bloqueos y ralentizaba la integración.

Identificamos esta mentalidad como el principal obstáculo para el avance. En las reuniones de [Retrospectiva](#) (Start Doing), se estableció la necesidad de una comunicación más proactiva. Se acordó utilizar el grupo de WhatsApp para comunicar los bloqueos y errores en tiempo real. Si un miembro tenía una solución, la compartía inmediatamente.

Esta regla progresó. Si una tarea en el Kanban excedía su estimación por varios días, y se hacía visible en el roadmap que llevaba abierta un tiempo considerable, cualquier integrante podía acoplarse a la tarea para ayudar, eliminando la noción de ‘propiedad’ y previniendo que se volviera un obstáculo bloqueante.

Otro riesgo muy presente durante todo el cuatrimestre fue la Disponibilidad del Equipo debido a la alta carga académica. Esto causó fricciones internas iniciales. En lugar de recurrir a la culpabilización, y echar en cara el que ciertos miembros no hubieran colaborado todo lo que deberían haberlo hecho (algo improductivo), la comunicación se priorizó en el [Sprint Planning](#). Cada miembro declaraba al inicio del sprint los exámenes, entregas y compromisos que afectarían su disponibilidad, permitiendo al equipo organizar la carga de trabajo de forma equilibrada y realista, mitigando así el riesgo de forma preventiva.

El Plan de Riesgos inicial, aunque minucioso, era un documento estático. Los problemas reales que surgieron, y que ya estaban contemplados en términos generales, se explicaban en los [Reviews](#), y las estrategias de Reducción y Supervisión se traducían en acciones de Start Doing o Stop Doing en las Retrospectivas. Para formalizar y activar la gestión, a raíz de las correcciones propuestas en el RTF, decidimos implementar [acciones correctivas](#):

- Creación del [Issue Tracker](#): Desde mediados del Sprint 5, cada incidencia (bug, error visual o funcional) se registró en el Issue Tracker, documentando su estado, fecha, prioridad y responsable. Esto transformó la Supervisión de subjetiva a cuantitativa y trazable.
- El Plan de Riesgos se convirtió en un Registro de Riesgos que se actualizaba obligatoriamente tras cada Retrospectiva. Se incorporaron las columnas de "Risk Owner" y "Métricas de Control", siguiendo las indicaciones del RTF. Esta implementación final aseguró que la gestión de riesgos no solo se realizara, sino que se demostrara, se midiera y se mejorara continuamente a lo largo del proyecto, adhiriéndose a la visión de un proceso de gestión ágil.

3.1.3. Ejemplos de Riesgos Principales y Abordaje

Los riesgos identificados como Intolerables y Altos se abordaron con planes de mitigación específicos integrados en las prácticas diarias del equipo:

3.1.3.1. Ejemplo 1: Cambio de lenguaje de C# a gd

A continuación se describe por etapas cómo abordamos el riesgo de cambiar de C# a GDScript.

1. Detección del problema

Durante las pruebas iniciales para la exportación web y publicación en itch.io, nos dimos cuenta de que el proyecto e ideas implementadas en C# no funcionaban al intentar pasar a HTML5. Aparecieron errores intentando usar las plantillas de exportación, tanto por la compatibilidad del lenguaje como por el propio comportamiento de Godot, lo que nos llevó a evaluar este como un riesgo significativo en el área de Arquitectura y Técnica.

De acuerdo a la gestión de riesgos que definimos, este problema que detectamos lo relacionamos con los siguientes riesgos:

- Falta de habilidades técnicas.
- Objetivos no alcanzados o desviaciones mayores por alta complejidad.
- Falta de investigación y documentación.

Por su alto impacto, el riesgo fue clasificado como **IN (Intolerable)**.

2. Evaluación de alternativas

Una vez identificado el problema, investigamos distintas opciones para intentar mantener el desarrollo en C#, con la idea de minimizar los cambios. Se probaron diferentes configuraciones de exportación, herramientas secundarias y cambios en el propio proyecto o Godot.

En esta fase es donde nos dimos cuenta de que el problema era real e irremediable fácilmente, y además podía provocar el estancamiento del proyecto y afectar negativamente al progreso de otros sprints.

3. Reconocimiento del error y toma de decisión

Tras analizar la severidad del riesgo, se asumió que no nos documentamos lo suficiente inicialmente y teníamos que actuar. Para evitar retrasos en tareas clave acumular más código que tener que pasar a futuro de lenguaje, se optó por abordar el problema de forma directa y definitiva, básicamente dándole entre dos compañeros todo un día de fin de semana hasta solucionarlo.

4. Implementación de la solución

Se decidió pasar el proyecto a **GDScript** y realizar la exportación a **HTML5** (lo que acepta [Itch.io](https://itch.io)). Se desarrolló el cambio en los siguientes pasos:

- Paso del código de C# a GDScript.
- Corrección de errores varios por el paso de lenguaje e incompatibilidad de funciones entre lenguajes.
- Remodularización de partes del código para mantener la coherencia previamente definida y asegurar mayor claridad para la revisión por parte del resto del equipo.

5. Reorganización del trabajo en el equipo

Para no ralentizar el desarrollo del resto del equipo y probablemente causar aún más errores, los apartados más específicos o menos conocidos por los responsables del paso de lenguaje se asignaron a los miembros del equipo con mayor conocimiento en esas áreas. Esta decisión permitió reducir el riesgo de contribuciones de baja calidad defectuosas.

6. Resultado y cierre

Al final el riesgo fue controlado y resuelto con un coste aceptable. Se aseguró que funcionase la exportación web, se evitó el bloqueo del proyecto para los demás y se protegió el avance del resto del desarrollo.

3.1.3.2. **Ejemplo 2: Multijugador web**

Otra dificultad que hubo que afrontar en el proyecto fue la exportación a formato web de la funcionalidad Multijugador, que abarca distintos Riesgos a la vez:

1. Detección del problema

En primer lugar, en el Sprint 4, cuando se implementó el modo multijugador, identificamos que no funcionaba en la versión web, por motivos desconocidos, lo cual se abarca en el riesgo de “Complejidad del modo multijugador persistente”.

Además, como no podíamos ver la pestaña de depuración en itch, no sabíamos qué fallaba, lo que hizo que la solución del problema se extendiese al Sprint 5, entrando en el riesgo de “Objetivos no alcanzados o desviaciones por alta complejidad”. Ambos riesgos se redujeron y supervisaron sin dificultad porque en depuración el multijugador era perfectamente funcional, así que no interfería con otras implementaciones.

2. Evaluación de alternativas

En este proceso, se reorganizó la prioridad de esta tarea y se descompuso en otras más asumibles progresivamente a medida que íbamos descubriendo y entendiendo el problema:

1. Conseguimos ver mensajes de depuración en itch e identificar el problema principal.
 - a. Solucionamos este primer problema, pero derivó en otro error distinto.
2. Consultamos la documentación de GDSync y solucionamos el error.
 - a. GDSync se iniciaba en itch correctamente, pero no podía conectarse a la IP que ofrecían los desarrolladores.
3. Nos encontramos ante un error por contenido mixto, que intentamos solucionar de distintas maneras:
 - a. Cambiamos itch por otros servidores web como Netlify, pero el error persistía.
 - b. Usamos un ordenador como servidor local http, pero aparecían tres errores distintos. Investigamos en el código html exportado y vimos que los 3 errores dependían de un error: “el servidor debía configurarse como https”.
 - c. Encontramos en el [repositorio](#) **GitHub de GDSync** un [issue abierto](#) y confirmado que describe exactamente nuestro problema, así que comentamos preguntando por avances.

3. Reconocimiento del error y toma de decisión (abandono)

En dicho issue se comentaba que el problema relacionado con ciertas políticas de los browsers y los desarrolladores están trabajando en posibles soluciones.

Como el problema persistía y no había soluciones potencialmente favorables, consideramos que habíamos derivado en un riesgo de “Requisitos que consumen mucho tiempo” (**IN (Intolerable)**). Finalmente decidimos que la relación entre esfuerzo y resultado potencial no era favorable y decidimos abandonar la tarea en favor de terminar otras.

En definitiva, la gestión de riesgos ha sido un **proceso dinámico** y fundamental para el éxito de Feudalia. La solidez del Modelo Boehm como marco inicial, combinada con la adaptación continua mediante las Retrospectivas, nos permitió transformar obstáculos en mejoras.

3.2. Revisiones Técnicas Formales (RTF)

La participación en las Revisiones Formales Técnicas (RTF) fue un mecanismo crucial para asegurar la calidad de nuestro proyecto, alineándolo con los estándares de la Ingeniería del Software. Este proceso se dividió en dos fases: la emisión de revisiones a otros equipos y la adaptación a partir del feedback que recibimos.

3.2.1. Definición de un Estándar de Revisión

Como equipo, nuestro primer paso fue establecer un estándar de calidad para uniformar las RTF que realizaríamos a los demás grupos. Este criterio no solo nos sirvió para ser objetivos a la hora de revisar la documentación y procesos de nuestros compañeros, sino también para saber qué debíamos exigirnos a nosotros mismos. Nos enfocamos en tres áreas:

- Criterios de Historias de Usuario (HU): Definimos que las historias debían ser uniformes, verificables y centradas en el usuario. Se puso especial énfasis en que los criterios de aceptación fueran medibles y concretos, una práctica que luego aplicamos a algunas de nuestras propias HU que aún no cumplían del todo con ella.
- Criterios de Gestión de Riesgos: Verificamos que se usara una matriz de Probabilidad e Impacto, y que hubiera planes claros de Reducción, Supervisión y Gestión para cada riesgo. También nos enfocamos en la trazabilidad (responsables) y la existencia de un mecanismo formal de actualización del plan.
- Criterios para Documentación Scrum: Revisamos que los documentos como el Sprint Planning y la Retrospectiva fueran útiles y que reflejaran una verdadera reflexión sobre procesos y colaboración, no solo avances.

Tras definir estos estándares, procedimos a redactar los informes para los tres equipos asignados, lo que nos proporcionó una visión externa de las debilidades comunes en la documentación. A continuación se enlazan las tres RTFs que hicimos al resto de equipos de la asignatura:

1. [Feedback para el equipo G2 \(Veneris\)](#)
2. [Feedback para el equipo G3 \(LogBait\)](#)
3. [Feedback para el equipo G4 \(Scrabble\)](#)

3.2.2. Implementación de Acciones Correctivas a partir del Feedback

La segunda fase, y la de mayor impacto para *Feudalia*, fue cuando recibimos y analizamos el feedback contenido en las [RTFs que nos entregaron los otros grupos](#). Estos informes señalaron áreas de mejora donde nuestra documentación y gestión, aunque existente, era ineficaz o desactualizada, lo que nos llevó a implementar cambios inmediatos:

A. Depuración y Mejora de las Historias de Usuario

El feedback de uno de los revisores (G2 - Veneris) identificó que nuestro documento de Historias de Usuario estaba contaminado con historias obsoletas o mal redactadas provenientes de los Sprints iniciales, lo que complicaba la trazabilidad en el backlog actual.

Plan de Acción: Acordamos un proceso de depuración progresiva, eliminando las Historias de Usuario en desuso y comprometiéndonos a revisar y reescribir a fondo todas las HU activas para que tuvieran una redacción clara y unos criterios de aceptación coherentes en cada Sprint.

B. La Gestión de Riesgos se Vuelve Activa

El informe de Revisión Técnica sobre nuestra Gestión de Riesgos (RTF de LogBait) nos indicó que nuestro Plan, aunque sólido, era un documento estático que no

usábamos de forma activa y que no tenía demasiado vínculo con el progreso diario del proyecto. Nos propusieron implementar cuatro acciones correctivas, y las asumimos todas:

1. Asignación de Responsables: Asignamos un "Risk Owner" para cada riesgo intolerable.
2. Definición de Métricas: Creamos "Métricas de Control" (como cuántos bugs arreglamos o la cobertura de pruebas) para medir si el riesgo estaba mejorando o no.
3. Trazabilidad y Control: Nos comprometimos a vincular cada riesgo con las Historias de Usuario o tareas específicas del backlog que se veían afectadas.
4. Actualización Continua: Decidimos que actualizar/revisar el Plan de Riesgos sería una actividad obligatoria al final de cada Retrospectiva.

Esta serie de medidas convirtió la gestión de riesgos en un proceso demostrable y medible, asegurando que las estrategias de Reducción y Supervisión se mantuvieran vivas a lo largo de todo el proyecto como ya se ha explicado en mayor detalle en el apartado 3.1).

Referencias: [Plan de Acción](#)

3.3. Conclusión sobre las RTF

Implementar estas medidas transformó nuestra forma de trabajar. Destacamos en particular el cómo pasamos de tener un plan de riesgos en un cajón a tener un Registro de Riesgos vivo y a usar el Issue Tracker para darle seguimiento a todo. La participación en las RTF fue lo que nos impulsó a formalizar estas herramientas de control, demostrando nuestra capacidad de adaptación y nuestro compromiso con la calidad del software.

4. Descripción del trabajo realizado

Aunque esta sección desglosa el trabajo por integrante, cabe destacar que el proyecto ha sido un esfuerzo eminentemente colectivo, tal y como promueve la metodología Scrum. Este enfoque ágil facilitó espacios de diálogo y cooperación vitales para el avance del desarrollo.

- En los Kanban de cada Sprint se puede consultar que gran parte de las tareas tienen comentarios activos por parte de todos los miembros del equipo.

Consideramos que métricas aisladas, como el conteo de commits o líneas de código, no son representativas del esfuerzo individual, dado el uso extensivo de prácticas como el Pair Programming, donde varios miembros trabajan sobre el mismo código simultáneamente. Además, el conteo de líneas de código no es representativo porque una parte significativa de Feudalia se programó gestionando nodos de distinto tipo, y esto no siempre se ve reflejado en las líneas de código.

Globalmente, estamos muy satisfechos con el desempeño del grupo y de cada uno de sus integrantes. Hemos completado el proyecto evolucionando juntos, pasando de ser un grupo de individuos a un equipo consolidado y familiarizado con la gestión ágil.

Andrés García-Redondo Gallego (Andy-uno):

Contribuciones y logros detallados por Sprint:

NOTA: La gran mayoría de commits realizados previos al Sprint 3 fueron junto a mi compañera Claudia, siendo yo el que subió los commits y por ende no está reflejado correctamente en GitHub la actividad de ambos esas primeras semanas. Esto se debe a no conocer las metodologías ágiles inicialmente y hacer tareas en conjunto, hasta hablar con el profesor en el Sprint 3.

SPRINT 1:

- Realización de la parte de especificación de requisitos de Feudalia, parte visual. Esto consistía en definir y dejar claramente redactadas las bases y diseños para Feudalia y/o generación de imágenes de referencia, además de búsqueda de asset packs para el juego y búsqueda de inspiraciones iniciales.
- Redacción de las funciones e ideas de Feudalia en la especificación de requisitos iniciales. Esto incluye: Vistas de ciudad, vista global, sistema de combate, sistemas de recursos y tecnología, el fin de partida y las funciones multijugador.
- Investigación de herramientas para el desarrollo de Feudalia.
- Aprendizaje del uso de godot (para posteriormente enseñar al equipo) y del uso de git para un proyecto de godot.

- Añadido addon de git para godot, usado para gestionar todo lo relativo a git desde la propia herramienta.
- Corregido y añadido cambios al README inicial.
- Primer prototipo:
 - Creación de un proyecto de godot y configuración inicial del mismo.
 - Creación, configuración e importación de los addons y carpeta de addons inicial.
 - Importación de la librería de assets “TinySwords” que sería la usada finalmente en todo el desarrollo (con añadidos).
 - Creación de estructura inicial del proyecto (escena main y menú).
 - Aprendizaje y creación sobre animaciones y sprites animados (player), junto con su código de movimiento y ataque iniciales.
 - Añadido y pruebas de fondos para el menu y escena principal.
 - Añadido de menú con botón “Play” funcional.
- Crear y organizar las historias de usuario para el primer prototipo
- Crear y organizar un primer Kanban fuera de GitHub y luego pasarlo todo a el actual Kanban del Sprint 1 tras hablar con el profesor.

SPRINT 2:

- Arreglos visuales y creación de una pantalla que sigue al personaje.
- Creación del primer recurso en el mapa (árbol).
- Creación del sistema de colisiones y pruebas con un primer recurso (árbol).
- Realización de pruebas unitarias de todo el prototipo inicial y colisiones.
- Primer prototipo del mapa: Investigación y creación del TileSet usado en Feudalia, configurando por primera vez los distintos tipos de atlas (“sheets de assets de casillas”) con colisiones. Esto incluye crear y probar las assets y colisiones de este mapa inicial
- Creación definitiva del primer cuadrante de Feudalia:
 - Creación de bocetos y diseño del cuadrante.
 - Re-importación de assets con sus colisiones para el nuevo terreno.
 - Creación de varias capas (TileMapLayers) ordenadas y apiladas para pulir gráficamente cómo se veía cada tile.
 - Creado a mano del primer cuadrante actual.
 - Creado y añadido de casillas animadas al mapa (efectos de agua principalmente).
- Crear y configurar Kanban del Sprint 2.
- Siguiendo las ideas iniciales de gestión de riesgos, tras el atasco de 3 compañeras en el intento de crear interacciones jugador-objetos, me uní y creé de cero el actual sistema de interacciones, dejando la base para solo configurar la reacción de cada objeto al ser golpeado.
- Pasado de documentos fuera de GitHub a la wiki actual.
- Corrección y arreglo de las historias de usuario iniciales del proyecto.

SPRINT 3:

- Creado del documento de estándares de diseño.
- Configuración y actualización del Kanban del Sprint 3.
- Realización de pruebas y arreglo de errores del proyecto a lo largo del sprint.
- Reordenación y refinado de historias de usuario.
- Creado de la UI de soldados:
 - Creado de botón sobre el edificio de reclutamiento (primer botón en el mapa propiamente) y programado del mismo.
 - Creado gráfico de parte de la interfaz.
 - Programado del código del menú de soldados y primeros botones interactivos inicialmente añadidos por Julia (responden al hover, pulsado...).
 - Creado y programado del gestor de soldados inicial (incluye los primeros usos de modularización definidos en los estándares de diseño), con las bases de interacción y modularización entre costes de compras (soldados y edificios) y el gestor de recursos.
 - Creado de la escena del soldado base para el combate.
- Añadidos al review del sprint anterior.
- Creación de documentos de ayuda (Guía de Git)

SPRINT 4:

- Desarrollo y redacción de la RTF al grupo “Veneris”.
- Creación de documento de control de calidad para otros grupos sobre las historias de usuario y actualizado del control de calidad propio.
- Creado de documento definiendo el PEGI 12.
- Para la subida online del proyecto, en Godot
 - Se investigó y llegó a la conclusión tras varios intentos de evitarlo de que era necesario pasar todo el proyecto de una versión de godot y lenguaje a otra.
 - Revisión y modularización de todo el código tras pasarlo a GDScript por problemas de exportación.
 - Arreglo de errores tras el cambio de lenguaje.
 - Añadido, configuración y primer uso de plantillas de exportación a HTML.
- Para la subida online del proyecto, fuera de Godot:
 - Creación de varios prototipos de página web para Feudalia (finalmente en desuso).
 - Creación de página directamente en [Itch.io](https://itch.io) con código en HTML, inicialmente más complejo, reducido a una versión pobre por problemas de permisos en la página (la actual versión).
 - Creación y edición del portal de administración en [Itch.io](https://itch.io) del proyecto subido en HTML como ejecutable web.
- Investigación para el multijugador. Se encontró finalmente tras varias ideas la herramienta GDSync, que permite manejar el multijugador actualmente, y sin la que

no se podría haber desarrollado toda esta funcionalidad. Se delegó el añadido e implementación del mismo a otros compañeros al ya haber tenido que hacer una gestión de tiempo por los errores en la subida a online.

- Editado del Sprint Review.
- Creación y gestión del Kanban.

SPRINT 5:

- Gestionado y corregido según la RTF recibida de nuestras historias de usuario.
- Editado del Sprint Review del sprint previo.
- Investigación y realización de pruebas para el multijugador.
- Creación y gestión del Kanban.
- Revisión completa de todas las funcionalidades del juego y creación de tareas de pulido o arreglo para cada error encontrado.
- Cambios varios y arreglos de bugs visuales.
- Implementación de áreas desbloqueables:
 - Finalmente se delegó la funcionalidad a los puentes, editando así los mismos en el mapa.
 - Implementado de botones de expansión (reconstrucción de puentes).
- Importado y añadido de todos los efectos de sonido actuales del juego.
- Investigación, importado y configuración inicial de la jugabilidad móvil.

SPRINT 6:

- Intento de creación de sonidos de pasos dinámicos según el tipo de suelo que se está pisando. Incluye la importación de otra librería de sonidos. Finalmente abandonado por problemas con funciones desactualizadas en la versión de godot usada y falta de tiempo.
- Creación y gestión del Kanban.
- Añadido de issues faltantes y edición del documento Issue tracker.
- Pulido y animado de la creación de los puentes.
- Añadido de transiciones entre escenas.
- Creación inicial de la estructura de esta misma memoria, desarrollo de varios apartados (Introducción, estructura y dos apartados de Scrum) y notablemente creación de todo el punto 4 de forma grupal, posteriormente eliminado.
- Añadido sfx de animación de los puentes
- Limpieza de archivos y cambios para la entrega final
- Creación y distinción de rama descargable y rama exportada a HTML para [Itch.io](https://itch.io), cambiando el menú para solo ofrecer lo que funciona en cada.

Carla Tomás Aguilar (C4rlaa):

Contribuciones y logros detallados por Sprint:

SPRINT 1:

- *Historias de Usuario:* Principalmente trabajé con Miriam en las historias de usuario previas, que como se comenta en el documento, finalmente fueron ocultas por su incorrección. Posteriormente elaboré otras que sí cumplían los requisitos, no obstante, quedaron en los documentos previos a la creación del repositorio, una vez decidido el nuevo enfoque basado en distintos usuarios.
- *Repositorio:* Organización inicial de carpetas, junto con un ReadMe primitivo.
- *Otra documentación:* contribuí con los primeros documentos de review y retrospectiva, junto a David (usando su equipo en clase).

SPRINT 2:

- *Historias de Usuario:* Especificación de alguna historia de usuario que se añadió.
- *Documentación:* Junto con Miriam, investigamos los ReadMe de otros proyectos de gitHub y diseñamos un modelo que consideramos óptimo de ReadMe. Posteriormente, trabajé en desarrollar aquellos aspectos generales, aunque importantes a conocer del juego en este documento. Tras algunos cambios de decisiones del equipo trasladé los cambios a los apartados de especificación de la wiki.

SPRINT 3:

- *Documentación:* al inicio del sprint redacté la review y la retrospectiva del sprint anterior además de preparar el backlog del kanban con las historias que habíamos hablado en conjunto.
- *Construcción:* añadí el botón de construcción, con su correspondiente menú desplegable, que de momento solo mostraba la vivienda junto a su precio. Añadí los cambios necesarios para los cálculos de compra y gestión de recursos. Finalmente, tras muchos intentos y problemas para que se ejecutase la previsualización y la construcción, Miriam consiguió detectar el fallo y terminar la construcción de viviendas.
- *Especificación:* hubo discordancias sobre la implementación de las viviendas y tras ponerse de acuerdo el equipo, lo ajusté en la especificación.

SPRINT 4:

- *Documentación:* redacción del sprint review del sprint anterior. RTF de las Sprint Reviews del equipo LogBait, añadiendo las sugerencias necesarias a la [wiki](#).
- *Modularidad:* modificación del código para cumplir los nuevos estándares de modularidad establecidos, así como la claridad y uniformidad de comentarios. Añadí una [plantilla a la wiki](#), por si ayudaba al equipo.
- *Pantalla final:* realización de la pantalla de resultados. Posteriormente traté de hacer unas mejoras en el formato, pero las horas de investigación e intentos no surtieron

efecto. Miriam ayudó con estos últimos errores. Más adelante esta pantalla se convirtió en una nueva escena.

- *Pruebas unitarias*: ejecuté pruebas y arreglé pequeños detalles del timer, la batalla y la construcción de casas.

SPRINT 5:

- *Matchmaking*: creé un script matchmaker para gestionar la recolección con los NPCs. Tras investigaciones de distintos métodos, y ensayos de implementación, evité subir avances que pudieran dañar el código funcional. Posteriormente Julia relevaría la tarea.
- *Pruebas unitarias*: realicé pruebas unitarias respecto al multijugador.
- *Testers*: me encontré con un par de testers externos al equipo que probaron el juego y comuniqué el feedback.
- *Multijugador en línea*: al ver que a Inés se le dificultaba la tarea de investigar alternativas para el multijugador en línea, propuse echar un cable. No duró mucho mi intervención, puesto que la tarea se consideró inviable.

SPRINT 6:

- *Documentación*: en busca de un refinamiento de cara a la entrega, hice detalles como actualizar el ReadMe, enlazar historias de usuario a la columna de backlog de cada sprint (tarea en la que ayudó Claudia) y últimas revisiones y retoques de la documentación de la wiki.
- *Batalla*: fue necesario hacer unas cuantas correcciones en el botón de llamada a batalla, respecto a funcionamiento y visibilidad y posteriormente a colisiones. Implementando esto encontré un error en los resultados, provocando que llevase a cabo una prueba unitaria.
- *Adaptación pantalla completa*: principalmente me encargué de ubicar todos aquellos elementos que no mantenían la posición relativa al cambiar el tamaño de la pantalla. No se trataban de cambios del layout. Recurrí a creación de nodos, herencia y distintas soluciones que investigué. No obstante, descubrí que en situaciones los errores venían de más lejos, por ejemplo una animación de la pantalla de inicio o del sceneManager en el caso del timer. Por lo que se convirtieron en cuestiones significativas en las que debí trabajar.

Claudia Villodre Pérez (cvillo01):

Contribuciones y logros detallados por Sprint:

NOTA: Debido a que en los primeros sprints no teníamos claro cómo funcionaba el desarrollo con metodologías ágiles, hice gran parte de las tareas con mi compañero Andrés ya que no sabíamos que debían realizarse individualmente. Es por esto, que las tareas realizadas no son acompañadas por commits desde mi cuenta, hasta el sprint 3.

SPRINT 1:

- Redacción del documento de especificación de Feudalia (parte visual). En este documento se especificó el aspecto visual del juego, creando un primer diseño de la interfaz, pensando en lo que tendría el juego (NPCs, edificios, etc) y por último buscando posibles juegos e imágenes de referencia.
- Redacción de las funciones o mecánicas del juego. Se incluyen las vistas de ciudad, la vista global, el sistema de combate, sistemas de recursos y tecnología, el fin de partida y el multijugador.
- Investigación de herramientas de desarrollo de videojuegos.
- Aprendizaje del uso de godot para posteriormente enseñar a los compañeros.
- Primer prototipo. En este primer prototipo aprendimos a crear un primer proyecto en godot y a configurarlo. Importamos los primeros assets, que al final hemos usado a lo largo de todo el proyecto. Creamos a nuestro protagonista, este contaba con un assets con diferentes animaciones (caminar, atacar, etc.) además del primer código que daba movimiento al personaje. Además, todo esto fue creado en nuestra primera escena que también contaba con un fondo estático que posteriormente sería el mapa. Por último, se hizo la primera estructura que contaba con pantalla de inicio funcional, que contaba con botón de play, y al darle cargaba la escena previamente definida.
- Organización de todos los archivos del proyecto (<code>)
- Crear y organizar las historias de usuario del prototipo
- Organización del primer Kanban, dejando ya la estructura para los demás.

SPRINT 2:

- Creación del sistema de colisiones. Se probó con un árbol, el primer recurso.
- Realización de pruebas unitarias de las colisiones.
- Comprobación por pruebas unitarias del primer prototipo creado en el Sprint 1.
- Primer prototipo de lo que es ahora el mapa. Incluyendo la investigación del uso de los TileSet y TileMap de Godot, usando estos por primera vez.
- Comprobación de las colisiones de este primer mapa.
- Creación definitiva del primer cuadrante de Feudalia:
 - Pensado y diseñado un primer boceto que hemos seguido.
 - Reimportación de assets con sus nuevas colisiones.
 - Creación de varias capas con niveles de visualización.
 - Creado a mano el primer cuadrante actual.
 - Creadas primeras animaciones del mapa (casillas animadas, como por ejemplo, efectos de agua).
- Crear y configurar el Kanban del Sprint 2.
- Pasar documentos hechos en estos dos Sprints a GitHub, por ejemplo, pasar la especificación de Feudalia a la wiki.
- Corrección de historias de usuario iniciales.

SPRINT 3:

- Configuración del Backlog y el Kanban.
- Realización de pruebas y arreglo de errores.
- Creado escena de la interfaz de soldados:
 - Creado menú de compra de soldados.
 - Creación del primer botón de edificios del juego.
 - Programado código de menú.
 - Programado primer código de botones de añadido de soldados. Esto implicó investigar cómo funcionaban los botones interactivos y hacer los primeros códigos usando la modularización.
- Creado mapa completo, que ha perdurado hasta el final (con revisiones puntuales):
 - Investigación de copias de secciones de un mapa y cómo invertirlas.
 - Detección y gestión de errores. Pensábamos que sería una tarea sencilla que nos quitaría poco tiempo. Al intentarlo nos dimos cuenta de que no funcionaba el girar los objetos y habían secciones del mapa que había que rediseñar enteras.
 - Rediseñé y redibujé los dos cuadrantes inversos.
 - Creadas y comprobadas todas las nuevas colisiones de los otros tres cuadrantes.
 - Creadas las murallas para la separación de los cuatro cuadrantes. Incluido un centro donde se encontrarían los botones para la batalla.
- Redacción de parte del review (sobre todo la parte referente al mapa).

SPRINT 4:

- Desarrollo y redacción de las RTF de las historias de usuario del grupo “Veneris”.
- Edición del Sprint Review.
- Creación y redacción documento de control de calidad para realizar las RTF de historias de usuario de otros grupos.
- Actualización del control de calidad.
- Revisión de todas las historias de usuario:
 - Eliminación de historias de usuario incorrectas o que ya no encajaban con nuestro objetivo de desarrollo.
 - Comprobación de concordancia con documento de especificación.
- Online:
 - Cambio de todo el código hasta el momento del proyecto de lenguaje. Para poder subir el proyecto a web, era necesario cambiar de lenguaje, por lo que pasé gran parte del tiempo convirtiendo el código de C# a GDScript.
 - Arreglo de errores tras el paso de lenguaje.
 - Subir proyecto a ich.io
- Arreglar el menú principal del juego.

SPRINT 5:

- Gestión y corrección de nuestras historias de usuarios tras recibir las RTF del otro grupo.
- Redactar/editar Sprint Review.
- Revisión completa de todas las funcionalidades del juegos y creación de tareas de gestión o arreglo de errores.
- Arreglo de errores: bugs visuales.
- Expansiones del cuadrante:
 - Diseño del mapa con puentes para poder gestionar las expansiones.
 - Creados modelos de puentes destrozados/construidos.
 - Creados botones de expansión.
- Investigación para implementar jugabilidad móvil.

SPRINT 6:

- Enlazado de historias de usuario en el backlog de cada Kanban.
- Pulido y animado de la creación de los puentes.
- Implementación a multijugador de las expansiones territoriales de cada cuadrante, lo que incluye arreglos en el mapa e implementación del código de los diferentes botones.
- Decoración:
 - Convertir assets de decoraciones a atlas para usarlos en el TileMap del proyecto.
 - Añadido de las decoraciones con cuadrantes a los cuatro cuadrantes.
 - Corrección de errores en el diseño del mapa.
- Pulido y finalmente borrado la jugabilidad móvil por exceso de errores y falta de tiempo.
- Desarrollo de apartados del documento de la memoria. Apartado 2.8.

David Morales Bravo (DavidMoralesGit):

Contribuciones y logros detallados por Sprint:

SPRINT 1:

- *Documentación:* Estructuración de la documentación del Documento de Especificación inicial, completando varios apartados acorde al estándar IEEE-830 y elaborando el Documento de Especificación de Requisitos (SRS) inicial.
- *Wiki:* Trabajo de especificación sobre primeros apartados de la wiki.
- *Historias de Usuario:* Diseño del User Persona 3 y redacción de algunas historias de usuario iniciales de este.

- *Mantenimiento y actualización de la documentación de los eventos Scrum:* Creación y redacción del primer Sprint Review Retrospective, junto con posterior modificación y revisión debido al inicio temprano de la fase de implementación.

SPRINT 2:

- *PMV:* Definición del alcance y redacción colaborativa del [Producto Mínimo Viable](#) (MVP) junto con Inés.
- *Sprites:* Creación e instanciación de sprites (casas, castillo), junto con edición y revisión de sus colisiones.
- *Pruebas unitarias:* Ejecución de pruebas unitarias sobre las primeras funcionalidades básicas.
- *Redefinición del concepto de juego:* tras cambiar de idea sobre el rumbo del juego, se modifica el trabajo hecho en el sprint anterior relativo a la especificación de requisitos en la wiki.
- *Historias de Usuario:* Redacción de nuevas historias de usuario por el cambio en la especificación y modificación de algunas anteriores.
- *Product backlog:* Elaboración y posterior refinamiento sobre el product backlog, creando también el kanban del sprint backlog y trabajando sobre la división en tareas de las historias de usuario asignadas a este periodo.
- *Documentación de reuniones de equipo:* Creación y actualización del [documento](#) en la wiki.

SPRINT 3:

- *Entorno:* Edición de la configuración inicial del mapa de juego (puentes o agua).
- *Menú de compra:* Implementación del sistema de construcción (menú desplegable y escenas para compra de edificios).
- *Product backlog:* Actualización de product y sprint backlogs y trabajando sobre la división en tareas de las historias de usuario asignadas a este periodo en el kanban.
- *Mantenimiento y actualización de la documentación de los eventos Scrum:* Confección de los sprint Planning, Review y Retrospective en colaboración con Miriam e Inés.
- *Control de calidad:* Revisión del documento añadiendo información acerca del PMV.
- *Gestión de riesgos:* Revisión de la documentación acerca de la planificación de riesgos de la wiki.
- *Documentación de reuniones de equipo:* Actualización del documento en la wiki con las daily meetings y la reunión de fin de sprint junto con Inés y Miriam.

SPRINT 4:

- *Documentación:* Creación y diseño del Documento Técnico (TDD), junto con posterior ajuste.
- *Arquitectura Multijugador y Red:* Implementación inicial del multijugador local e integración del plugin GD-Sync con el código.

- *Reorganización lógica de red*: Optimización de la sincronización de los modos de juego y gestión de la concurrencia.
- *Product backlog*: Actualización de product y sprint backlogs y trabajando sobre la división en tareas de las historias de usuario asignadas a este periodo en el kanban. Aplico también las métricas de estimación especificadas en la wiki.
- *Control de calidad*: Ejecución de las RTF de uno de los grupos compañeros (Logbait) sobre su Sprint Planning, añadiendo el feedback dado a la wiki.
- *RTF*: Análisis del feedback recibido y gestión de los cambios identificados.
- *Documentación de reuniones de equipo*: Actualización del documento en la wiki con las daily meetings y la reunión de fin de sprint junto con Inés y Miriam.
- *Mantenimiento y actualización de la documentación de los eventos Scrum*: Trabajo sobre el sprint planning y retrospective junto con Inés.

SPRINT 5:

- *Product backlog*: Actualización de product y sprint backlogs y trabajando sobre la división en tareas de las historias de usuario asignadas a este periodo en el kanban. Aplico también las métricas de estimación especificadas en la wiki.
- *Implementación del flujo de navegación*: Configuración del menú inicial, selectores de modo de juego y gestión de escenas.
- *Arquitectura Multijugador y Red*: Corrección de conflictos críticos en la ejecución simultánea de los modos PvE y PvP en entorno local, con la gestión de desconexiones y demás incidencias junto con Inés.
- *PVE*: Diseño y programación de la lógica del Bot para el modo PvE, incluyendo la corrección de errores en su sistema de construcción.
- *Mantenimiento y actualización de la documentación de los eventos Scrum*: Trabajo sobre el sprint planning y retrospective junto con Inés.
- *Pruebas unitarias*: Ejecución y documentación de pruebas unitarias sobre funcionalidades pendientes.

SPRINT 6:

- *Documentación*: Revisión de la documentación de todo el proyecto (wiki) de cara a la presentación final, y edición de soporte visual en Canva para presentación.
- *Product backlog*: Actualización de product y sprint backlogs y trabajando sobre la división en tareas de las historias de usuario asignadas a este periodo en el kanban. Aplico también las métricas de estimación especificadas en la wiki.
- *Experiencia de usuario*: Desarrollo del sistema de carga (pantalla de carga con animaciones) y su sincronización entre los distintos modos y escenas de juego. Depuración general de errores (debugging) en la interacción del usuario con el entorno.
- *Gestión de riesgos*: Revisión y actualización del registro de incidencias (Issue Tracker).

Inés Pérez Herrera (inespe10):

Contribuciones y logros detallados por Sprint:

SPRINT 1:

- *Documentación:* Investigué y organicé la documentación del Documento de Especificación acorde al estándar IE-830, añadiendo las secciones faltantes.
- *Wiki:* Creación de un índice inicial de la Wiki con Miriam y traslado de las secciones 1 y 2 del documento de especificación a la misma.
- *Dibujos iniciales:* Dibujé las mecánicas iniciales para unificar visiones.
- *Historias de Usuario:* Diseño del Usuario 1 estereotípico y redacción de algunas historias de usuario iniciales (se borraron por las RTFs y cambios de especificación).
- Añadí todas las historias de Usuario de la Wiki al [Product Backlog](#).
- Redacción del apartado de [diseño gráfico](#) de la Wiki y de la [Pantalla Home](#).

SPRINT 2:

- *Implementé la ordenación YSort* en los elementos correspondientes.
- *Animación de los árboles:* Andrés añadió animaciones a los árboles, pero no se reproducían correctamente. Identifiqué que no se ejecutaban porque habíamos cambiado el nombre de varios ficheros en Godot y no se habían actualizado al hacer commit-push. Ajusté manualmente esto desde GitHub.
- *Recolección de recursos:* Creación de nodos con sus respectivas colisiones. Traslado de la mecánica (implementación de Andrés) a las piedras y las minas. Añadí una capa de recursos al main y todos los recursos a un TileMap Layer.
- *Colisiones:* Ajusté las colisiones de los elementos del mapa acorde al TileMap.
- *PMV:* Redacté con David el documento del [Producto Mínimo Viable](#).
- *Pruebas unitarias:* Redacté el documento de [pruebas unitarias](#) y redacté los tests que hacían falta hasta el momento ([Funcionalidades básicas](#) 1-5).

SPRINT 3:

- *Implementación de los recursos:* Rediseñé las colisiones de las piedras.
- *Crecimiento de la población lineal:* Implementé que la población crezca de forma lineal al añadir una casa.
- *Configuración del Kanban:* Y añadido de estimates y fechas de inicio-fin en las tareas en las que no estaba especificado.
- *Organización de documentación Scrum:* Ordené los documentos de Retrospectiva y Revisión, por estar conceptualmente mezclados y añadí a estos, la planificación, el backlog y el sprint backlog un índice de contenidos de guía para hacer bien esta documentación en adelante.
- *Redacción del documento de [Técnicas de Estimación](#)* y cambié el Product Backlog para introducir los estándares nuevos.

- *Otros documentos:* Redacté el Sprint Planning (4), mi parte del Review y la Retrospectiva.

SPRINT 4:

- *Batalla:* Creé los nodos de las tropas y del campo de batalla en un nodo separado (para evitar solapamientos con el main). Cambié la estructura del botón de batalla a una escena y lo añadí en el resto de cuadrantes.
- *Modo multijugador (tareas):* Andrés y Claudia encontraron el plugin GDSync. David y yo reajustamos las tareas del Kanban para implementarlo.
- *Modo multijugador (cuadrantes):* Asignación de la posición inicial en función del cuadrante y modo de juego, asignación de botones (batalla y reclutamiento) por cuadrantes.
- *Modo multijugador (comunicación):* Investigación de formas de conectar datos entre dos jugadores en un mismo lobby. Coordiné la comunicación del número de tropas entre jugadores, para lo cual tuve que solucionar varios problemas “de carrera” que había en la implementación presente.
- *Implementación de un Timer:* Visualización de un temporizador con mensajes y cambios de color (tarea solapada con Julia, solucionamos juntas las diferencias de implementación).
- *Ajustes por el cambio de gd:* Repetí el modo multijugador y el timer.
- *Configuración del Kanban:* Y añadido de estimates y fechas de inicio-fin.
- *RTF de planning, review y retrospective:* Redacté los [criterios de corrección](#) de este apartado. Hice la [RTF](#) de la retrospectiva.
- *Otros documentos:* Redacté el Sprint Planning (5), mi parte del Review y la Retrospectiva y el Sprint Backlog con David.

SPRINT 5:

- *Reorganización del código* en carpetas claras y eliminación de código obsoleto.
- *Multijugador (pulido):* Gestión de caídas (si tu enemigo se cae, cambias automáticamente a modo PVE y puedes terminar la partida), partidas simultáneas (se crean distintos lobbys numerados para soportar partidas simultáneas), duplicado del inicio del plugin, coordinación del Timer entre jugadores (disminuí el delay), hice que las casas fueran locales a cada jugador y gestión de dos modos de juego (Miriam lo implementó y yo solucioné problemas con el enemigo automático).
- *Lógica de reseteo de partida:* Ajusté el botón existente para que resetease los autoloads del proyecto, se gestionasen los lobbys del multijugador y solucioné problemas de carrera derivados.
- *Estética de la pantalla de carga:* Añadí las animaciones, generando los sprites por separado a partir de gifs que encontró Julia.
- *Multijugador web:* Identificación de errores, causas y posibles soluciones. Apliqué la Gestión de Riesgos para intentar solucionarlo (véase la sección [3.1.3](#)).

- Cambio en la configuración de la exportación web para que GDSync se pudiera iniciar en un entorno web.
- Cambio en el inicio de GDSync para conectarse a una IP.
- *Intentos de solucionar el problema por contenido mixto*: intenté subir el juego a Netlify, Poki y CrazyGames obteniendo el mismo error.
- *Prueba de depuración con servidor local*: Usé mi ordenador como servidor local, pero obtuve un error que decía que “el servidor debía configurarse como https”.
- *Reimplementación del multijugador*: La pantalla de carga añadía un delay de más de 2 segundos. Rehice la lógica con David y repetí las reconexiones y caídas.
- *NavigationMaps*: Añadí el NavigationMap del segundo cuadrante y lo reflejé al primero.
- *Configuración del Kanban*: Y añadido de estimates y fechas de inicio-fin.
- *RTF de documentos SCRUM*: Redacté el [feedback](#) de Scrabble sobre nuestra documentación SCRUM.
- *Otros documentos*: Redacción de mi parte del Review, y el Sprint Planning (6),

SPRINT 6:

- *Multijugador web*: Investigación en foros de GitHub (véase la sección [3.1.3](#)).
- *Vista del juego*: Ajusté que fuera proporcional al tamaño de la pantalla.
- *Solución de warnings y errores de depuración* (nombres de variables y rediseño de la lógica de implementación de los botones de batalla).
- *Exportación para Windows y Linux* del juego con sus instrucciones.
- *Cambio de color de los personajes*: Añadí las escenas, lógica y sprites necesarios en el personaje principal y las tropas del campo de batalla.
- *Añadido de estadísticas*: Implementé la lógica de partidas “ganadas”, “perdidas” y “empatadas” que aparecen en la pantalla de resultados.
- *Añadido de mensajes para el jugador*:
 - En PVP, si tu enemigo se cae, puedes ver un mensaje en pantalla que te informa de que has cambiado de modo.
 - Si no pudiste conectarte a GDSync, no recibiste señal del lobby (ocurre con la red de la UCM) o no se encuentra contrincante en 30 segundos lo notifica.
- *NavigationMaps*: Añadí lógica para que los mapas fueran dinámicos y se adaptasen a los puentes según si estaban contruidos o no.
- *Configuración del Kanban*: Añadido de estimates y fechas de inicio-fin. Eliminación de tareas duplicadas por el gestor automático de GitHub.
- *Añadido de tests unitarios*: añadido de los tests [Funcionalidades Avanzadas](#) y [Otros](#).
- *Presentación del proyecto*: Diseño de mis diapositivas de la presentación, actualización del Readme final y redacción de la memoria (sección 5 y 3.1.3.2).
- *Otros documentos*: Redacción de mi parte del Review.

Julia Ceregido Andrade (julcereg):

Contribuciones y logros detallados por Sprint:

SPRINT 1:

- *Diseño de Usuarios:* Creación de las User Persona 2 y 4.
- *Redacción de las Historias de Usuario:* Aquellas asociadas a las User Persona 1 y 4, estableciendo los requisitos funcionales iniciales para Feudalia.

SPRINT 2:

- *Lógica de Recolección Manual:* Implementación de la mecánica de recolección, incluyendo la lógica de tres golpes, el tiempo de regeneración específico de cada recurso y la correcta actualización del Resource Manager. El rediseño de la lógica de un único hitbox en el personaje principal en vez de en cada recurso individual fue cosa de Andrés.
- *Visualización de Recursos:* Desarrollo de la implementación visual completa de la recolección. Esto incluye la codificación de las animaciones de ser golpeado/talado/minado, las animaciones del estado agotado (depleted) y las animaciones auxiliares para los sprites indicadores (tronquito, bolsa de oro, etc.).
- *Documentación de Gestión y Riesgos:* Redacción del documento Definition of Done (DoD) y de la Introducción de Riesgos (Parte 1) y el Desarrollo de Riesgos Principales (Parte 3) del Plan de Gestión de Riesgos. Los otros apartados dentro de la gestión de riesgos fueron obra de Miriam.
- *UI:* Selección de los iconos para cada recurso utilizado en el juego.

SPRINT 3:

- *Diseño e Interfaz de Reclutamiento:* Creación del interfaz de menú de reclutamiento de soldados, incluyendo la lógica de aparición/desaparición al pulsar el botón del cuartel/castillo.
- *Lógica de Compra y UX:* Implementación de la lógica de compra de soldados (verificación de recursos suficientes) así como del feedback visual de botones en rojo cuando no hay recursos para la compra, y creación de los Tooltips para mostrar el precio unitario de cada soldado.

SPRINT 4:

- *Arquitectura Central (GameState):* Creación e implementación del autoload GameState, crucial para llevar registrado el estado de la partida (estado de conexión a pve/pvp, estado de recolección, estado de posible ataque, estado de batalla, etc) a través de las diferentes escenas. Este timer GameState que hice sería reutilizado para gestión de escenas, ya que el estado de partidas según tiempos se implementaría al final en el timerRoot, por Inés.

- *Gestión de Calidad:* Redacción de la mayor parte del documento de Gestión de Calidad, estableciendo el marco de procedimientos de inspección, revisión y pruebas según los estándares definidos en clase.
- *Lógica de Declaración de Batalla:* Desarrollo del botón de ataque en el mapa, incluyendo la lógica de visibilidad por zona de detección, modo disabled durante el estado de recolección, feedback visual (cambio a rojo) y el transporte al campo de batalla con las tropas actuales.
- *Visualización de Combate:* Implementación de la visualización del ejército al inicio de la batalla, asegurando que las tropas se instancien con la cantidad exacta, separadas y mirando al centro en filas por tipo de unidad. El que los ejércitos batallaran tras aparecer no lo hice yo sino Miriam.
- *Reglas de Construcción:* Revisión de la construcción para evitar que se pueda construir encima de recursos (rocas, árboles, minas) o sobre el personaje principal.
- *Otra Documentación:* Redacté las RTF de otro equipo (Scrabble) y actualicé nuestro propio Plan de Gestión de Riesgos basado en el feedback obtenido de las RTF que hicieron a nuestro proyecto.

SPRINT 5:

- *Metodología:* Creación y organización del Kanban, así como redacción de parte del review del Sprint 4.
- *Diseño de Oficios y NPCs:* Diseño de las escenas físicas de los tres tipos restantes de casas (de oficio) y de las escenas completas, con animaciones, para cada tipo de NPC recolector.
- *Implementación de Recolectores:* Implementación de la funcionalidad del botón de compra y el spawn de los NPCs recolectores, incluyendo la corrección para asegurar que solo spawnearan en terreno válido.
- *Investigación de IA:* Investigación de posibles algoritmos para la búsqueda de recursos por los NPCs recolectores.
- *Documentación:* Redacción integral del Manual del Usuario/Jugador.

SPRINT 6:

- *Metodología:* Organización del Kanban, así como redacción de parte del review del Sprint 5, la redacción completa del Sprint (6) Backlog y del Sprint (5) Retrospective.
- *Lógica de Batalla y Resultados:* Implementación de una escena separada para mostrar los resultados de la batalla y corrección de la lógica de batalla para añadir la opción de empate.
- *Presentación:* Elaboración y diseño de las diapositivas del Power Point (Interfaces, Recolección, Construcción, NPCs y Estadísticas).
- *Mantenimiento y Estabilidad del Código:* Corregí dos veces el comportamiento del botón de batalla y la animación de cuenta atrás que habían dejado de ser funcionales tras otras modificaciones, asegurando la estabilidad de la mecánica de ataque. (Implementé la animación de cuenta atrás hasta cuatro veces debido a constantes regresiones).

- *Documentación:* Creación y llenado del Issue Tracker con problemas con los habíamos ido topando. Revisé las historias de usuario para asegurar el formato adecuado. Redacté el Retrospective completo del Sprint 5.
- *Implementación de Recolección por NPCs:* Implementación del código de recolección de recursos por NPCs, la tarea de IA más compleja del proyecto:
 - Uso del NavigationAgent para que los NPCs busquen su recurso más cercano.
 - Lógica de exclusión para que solo un NPC pueda ir a un recurso, forzando al otro a buscar una alternativa.
 - Implementación del comportamiento de recolección idéntico al manual (animación, depleted, suma al Resource Manager).
 - Mecanismo de búsqueda de nuevo recurso si el camino al objetivo inicial está bloqueado tras un tiempo estipulado.

Miriam Elena Cheikho Ferariu (mch-boop):

Contribuciones y logros detallados por Sprint:

SPRINT 1:

- *Wiki:* Creación de un índice inicial de la Wiki con Inés.
- *Historias de Usuario:* Creación del User Persona 3, y elaboración de sus respectivas historias de usuario, asociadas al perfil y a los requisitos del sistema. Además, trabajé en la diversificación de los usuarios del proyecto, para tener perfiles variados y realistas, elaborando ilustraciones propias para cada uno, para evitar problemas de copyright.

SPRINT 2:

- *Interfaz y Sistema de Recursos:* Creación de la interfaz de recolección de recursos, incluyendo la búsqueda y adición de iconos representativos para cada recurso. Además, participé en la implementación del código de recolección, así como en el desarrollo del sistema de actualización de recursos posterior, trabajando en coordinación con Andrés, Julia e Inés.
- *Historias de Usuario:* Identificación y adición de nuevas historias de usuario necesarias para reflejar correctamente las funcionalidades implementadas durante el sprint.
- *Documentación:* Elaboración del archivo README del proyecto junto a Carla, asegurando una explicación clara del funcionamiento general y de los aspectos técnicos más relevantes. Asimismo, realicé la clasificación de riesgos del Plan de Gestión de Riesgos (parte 2), contribuyendo a la identificación y organización de posibles riesgos del proyecto.

SPRINT 3:

- *Construcción:* Colaboración en el diseño del menú de construcción y apoyo en la corrección del funcionamiento de la construcción de casas, logrando que el sistema funcionara correctamente.
- *Sistema de Recursos:* Añadí límites máximos al contador de recursos y pulí el sistema de recolección y actualización de recursos desarrollado en el sprint anterior
- *Historias de Usuario:* Creación de historias de usuario relacionadas con las mecánicas de combate.
- *Documentación:* redacción de la planificación temporal y el resumen (partes 4 y 5) de la gestión de riesgos del proyecto.

SPRINT 4:

- *Combate:* Participé en la resolución de los problemas de visibilidad del botón de ataque, asegurando que la interacción fuera clara para el jugador. Además, junto a Inés, añadí un segundo botón de ataque en el cuadrante correspondiente al segundo jugador, permitiendo una correcta interacción de cara al modo multijugador.

También me encargué de la programación del sistema de combate, haciendo que ambos ejércitos se desplazaran hacia el centro del campo de batalla y ejecutaran la animación de ataque al entrar en contacto. Finalmente, trabajé en la lógica del fin de batalla, comparando el número de tropas de cada ejército tras el enfrentamiento para determinar el resultado.

- *Construcción:* Retoqué y pulí el sistema de construcción de casas para evitar errores durante la colocación de edificios. En concreto, revisé que no se permitiera construir en zonas con colisiones (paredes, rampas, agua u otros edificios) y que el sistema indicara correctamente, mediante el cambio a color rojo, todos los casos en los que la construcción no era posible, ya fuera por colisión o por falta de recursos. Asimismo, corregí errores derivados del paso a GD en la construcción de casas, mientras que el resto de ajustes fueron abordados por otros miembros del equipo.
- *Gestión de Riesgos:* Definición de los criterios para las RTF de Riesgos a seguir a la hora de realizar la RTF del equipo correspondiente.
- *Interfaz:* Apoyo en el formateo de la pantalla de fin de partida, destinada a mostrar si el jugador había ganado o perdido, así como información sobre el adversario, aunque posteriormente esta pantalla fue sustituida por otra escena.

SPRINT 5:

- *Documentación:* Redacté el Sprint Backlog del sprint 5, organizando todas las tareas y objetivos planificados para el sprint, y también redacté la Sprint Retrospective del sprint 4, recopilando aprendizajes, dificultades y mejoras para los siguientes sprints.
- *Colisiones y Mapas:* Colaboración en la revisión de las colisiones de los edificios de recolección y su correcta visualización con respecto al resto del mapa, asegurando que

la interacción de los jugadores con los edificios fuera consistente y que no se produjeran errores visuales o lógicos.

- *Modos de Partida:* Participé en la gestión de modos de partida, incluyendo la configuración de la Batalla con Enemigo Automática y la lógica de inicio del enemigo automático. Además, revisé y creé historias de usuario relacionadas con los modos de partida, garantizando que todas las funcionalidades estuvieran correctamente definidas y alineadas con los requisitos del proyecto.
- *NPCs:* El spawn aleatorio de NPCs alrededor de sus casas ignoraba colisiones, lo que podía provocar que se generaran sobre edificios, agua o recursos. Esto se solucionó, y llevó al inicio de la tarea, que se continuó en el sprint siguiente, de implementar colisiones y la recolección para cada tipo de NPC, gestionando sus movimientos, colisiones y animaciones. Como parte de esta preparación, también se inició la creación y ajuste de los NavigationMaps en los cuadrantes, que sirvieron de base para la correcta navegación de los NPCs en el mapa.

SPRINT 6:

- *NPCs y movimiento automático:* Avance en la implementación de colisiones y recolección de los leñadores, ajustando los NavigationMaps para garantizar un comportamiento correcto de todos los tipos de NPC en el mapa. Estuve trabajando en su movimiento correcto y que localizaran la ruta más adecuada, evitando colisiones. Implementé además la lógica de recolección de los recolectores restantes (mineros y canteros) una vez conseguí arreglar el movimiento de los primeros.
- *Colisiones:* revisión de las colisiones de los cuatro cuadrantes para asegurar la correcta interacción con el entorno.
- *Campo de Batalla:* Modifiqué la estética del campo de batalla, ajustando el suelo para mejorar la claridad visual y la experiencia de juego.
- *Historias de Usuario y Documentación:* Revisé y ajusté las historias de usuario según las nuevas funcionalidades implementadas. Redacté el Sprint Backlog del sprint 6 junto a Julia, colaboré en la creación del Issue Tracker y actualicé el README para reflejar los últimos cambios del proyecto.

5. Código

El código fuente completo de Feudalia se encuentra en el repositorio GitHub de AlgoRitmos:

- <https://github.com/UCM-FDI-DISIA/proyectois1-algoritmos>

En particular, la entrega completa, revisada y funcional, queda registrada en la última Release del equipo, que incluye lo siguiente:

- El [proyecto completo](#) de Godot (fuentes, escenas, recursos, plugins, etc).
- Los [ejecutables](#) exportados para Windows y Linux, para probar el juego sin depender del entorno de desarrollo.
- Un fichero [README.md](#) con las instrucciones de configuración, exportación (lo más parecido a compilación) y ejecución. También incluye una presentación al juego.

Nota técnica: en Godot 4.5. y su lenguaje nativo, GDScript, no existe una compilación tradicional como en C o C++, el flujo habitual de desarrollo es importar el proyecto en Godot y ejecutarlo, o bien exportarlo a distintos tipos de ejecutables.

En términos de organización y mantenibilidad, el proyecto comenzó sin una estructura definida, pero, gracias al diálogo que abrieron distintas Revisiones, decidimos fijar una serie de [Estándares de Diseño](#) con los siguientes objetivos en mente:

1. Asegurar la modularidad del código de cada Script, especialmente en los gestores.
2. Mejorar la legibilidad del código: de esta forma, cada desarrollador podía trabajar sobre código ajeno de manera más sencilla.
3. Facilitar la modificación de constantes, parámetros, mensajes de texto...

Organización del repositorio:

Nuestro repositorio GitHub está organizado en las siguientes carpetas:

- *addons*: Plugin GDSync y SceneManager.
- *executable*: Ejecutables para Windows y Linux con instrucciones detalladas sobre el uso de Feudalia, como ejecutarlo y una visión general del entorno de desarrollo en un [fichero aparte](#).
- *network*: Gestión del plugin. Aquí se encuentran los ficheros programados por el equipo para la gestión del uso del plugin GDSync.
- *sfx*: Recursos sonoros y librerías de sonidos.
- *src*: Código y lógica del juego. Esta carpeta, a su vez, se divide en distintas secciones, como botones, pantalla principal, pantalla de carga, campo de batalla, NPCs, recursos, edificios, assets, menús...
- *tests*: Pruebas unitarias y de integración.

5.1. “Test” unitarios

Además, otro pilar fundamental en la programación de nuevas funcionalidades (tanto por código como por gestión de nodos) han sido los **tests unitarios**. En nuestro caso, el uso de Godot con GDScript nos impidió realizar tests unitarios automáticos al uso, así que decidimos decantarnos por tests unitarios manuales.

Estos tests unitarios siguen un formato y unos [estándares](#) rigurosos. Todos ellos incluyen un título, pasos a seguir para realizar la prueba, resultados esperados y resultados obtenidos (se registran en caso de incidencia con la fecha de realización de la prueba). Bajo nuestros Estándares de Pruebas Unitarias, procuramos que los tests unitarios cumplan lo siguiente:

Nuestro método de trabajo integra los tests unitarios de manera que cada programador redacta los tests unitarios de la funcionalidad que va a desarrollar antes de implementarla (al menos el resultado esperado). Después, cuando la nueva mecánica se ha podido integrar al juego, se revisa la corrección del test y se añade a la carpeta designada para ello del repositorio.

No obstante, encontramos que estos tests no eran suficientes para asegurar el progreso del equipo. A partir del Sprint 5 aproximadamente, cuando ya se habían introducido varias funcionalidades complejas, vimos que cambios mínimos introducían errores que rompían funcionalidades anteriores y que eran difíciles de registrar o ubicar en los tests, ya que conectaban varias funcionalidades de formas variadas y complejas. Además, esto estaba dificultando la comunicación del equipo y el flujo de trabajo, porque el avance de cada desarrollador se veía condicionado por errores cuya causa desconocía.

Por este motivo, siguiendo la Gestión de Riesgos correspondiente, decidimos introducir un [Issue Tracker](#), complementario al proceso de Pruebas Unitarias. El Issue Tracker recopila los errores mayores que ocurren en la ejecución del juego y asigna responsables para solucionarlos. De esta manera, se buscó que el proceso de desarrollo no se viera interrumpido y que, en la medida de lo posible, no se solapasen errores de distinta naturaleza, para no complicar innecesariamente su solución.

Todo ello en conjunto mejoró la trazabilidad del proceso

bug → corrección → verificación.

5.2. Uso de IA generativa

El uso de IA generativa en el desarrollo de Feudalia ha sido muy variado a lo largo del proyecto Feudalia, adaptándose a las necesidades específicas de cada funcionalidad.

Entre las herramientas utilizadas se encuentran Chat-GPT, Copilot, Kimi o LMArena. Cabe destacar el uso de un modelo de Chat-GPT entrenado con la documentación técnica del proyecto, lo que nos permitió obtener respuestas más contextualizadas y alineadas con la arquitectura y convenciones de codificación de Feudalia.

Principalmente, podemos distinguir los siguientes usos:

- *Manejo de Godot*: Consulta de funcionalidades específicas del motor, comprensión de la arquitectura de nodos y escenas, uso de recursos nativos (AnimationPlayer, NavigationMaps, TileMaps, YSort...) y navegación por los menús de configuración de proyecto. Ídem para los plugins.
- *Programación en C# y GDScript*: El desconocimiento inicial de los dos lenguajes en los que se ha desarrollado el proyecto hizo imprescindible el apoyo de IA para comprender la sintaxis, estructuras de datos específicas, manejo de señales... También nos facilitó significativamente el cambio de C# a GDScript, que de otra manera hubiera sido inviable.
- *Resolución de problemas*: Depuración de errores en tiempo de ejecución, identificación de problemas de rendimiento (delays), solución de conflictos entre distintas componentes, corrección de comportamientos inesperados en la lógica del juego y en las interacciones entre nodos.
- *Investigación sobre posibles implementaciones*: Exploración de diferentes aproximaciones técnicas para resolver un mismo problema, evaluación de plugins externos (como GDSync), investigación sobre algoritmos específicos para una determinada tarea...
- *Comprensión de errores de depuración*: Interpretación de mensajes de error y warnings del motor, análisis de sus respectivas stack traces (especialmente cuando el error lo lanza un Script interno de Godot o GDSync), identificación de null reference exceptions (y por qué se dan), identificación de errores de carrera...
- *Generación de algoritmos estandarizados*: Implementación de algoritmos pathfinding (como A*), sistemas de generación de estados y managers...

Por este motivo, la gran mayoría de Scripts de código se han creado con apoyo de IA generativa en mayor o menor medida. En cuanto a código generado por IA per se, cabe destacar la implementación de la conexión entre jugadores a través de GD-Sync y la implementación del algoritmo A* como los Scripts con mayor porcentaje de código escrito por IA. No obstante, aclarar que todo el código ha requerido de modificaciones, ajustes, funciones auxiliares y revisiones humanas para la integración con el entorno existente.

Por otro lado, en relación al diseño de prompts, a lo largo de cada Sprint fuimos perfilando como equipo cómo podíamos utilizar la IA generativa para que el progreso fuera lo más lineal y limpio posible. Gracias al diálogo que ocasionaron varias Retrospectivas, vimos la importancia de utilizar la IA de forma consciente y planificada, con prompts elaborados que pudiesen asegurar que no se iba a romper código existente.

A lo largo del cuatrimestre hemos aplicado distintos principios para generar prompts efectivos:

1. Contexto completo:
 - a. Descripción clara de la funcionalidad deseada y el estado actual.
 - b. Inclusión de herramientas, versiones, plugins y arquitectura.
2. Especificidad técnica:
 - a. Lenguaje de programación, estructura de nodos, convenciones de estilo.
 - b. Pseudocódigo o lógica esperada.
3. Control sobre modificaciones: Si se trabaja sobre ficheros existentes añadir frases del tipo “no modifiques ninguna línea funcional ni los métodos existentes”.
4. Inclusión de errores: Errores reales, contextos, intentos previos...

Ejemplos de prompts:

Ejemplo de Prompt de generación de código:

Estoy desarrollando un juego en Godot 4.5. Tengo una escena principal “main”, un subnodo StaticBody “Arbol” con una animación por defecto “idle” y un subnodo CharacterBody “Player” movable por flechas, ambos con colisiones. Player tiene una animación de ataque ligada a un booleano “is_attacking”, quiero que cuando el jugador ataque a Arbol, Arbol cambie de animación a “hit”. ¿Cómo lo hago? Explicame paso a paso cómo crear y configurar los subnodos que necesitan Player y Arbol y cómo programar en C# las señales entre nodos con funciones independientes para esta tarea. Añade esas funciones a este código, sin modificar las funciones que ya tienen: [código del árbol] [código de Player].

Ejemplo de Prompt de solución de errores:

He desarrollado un juego en Godot 4.5 con un modo multijugador que depende del plugin GD-Sync y un modo individual independiente, ambos modos funcionan perfectamente al ejecutarlo en la ventana de depuración de Godot.

Lo he exportado a versión web para subirlo a itch y he comprobado que el modo individual funciona, pero al intentar ejecutar el multijugador encuentro que el plugin se inicia pero me da un error de contenido mixto:

```
“was loaded over HTTPS, but attempted to connect to the insecure WebSocket endpoint 'ws://64.225.79.138:8090/'. This request has been blocked; this endpoint must be available over WSS.”.
```

He intentado solucionarlo escribiendo wss antes de la IP para “forzarla” a ser segura, pero entonces no la reconoce. ¿Cómo puedo evitar el conflicto de contenido mixto sin modificar la IP que me ofrecen los desarrolladores?

Ejemplo de Prompt de uso de Godot:

Estoy desarrollando un juego en Godot 4.5. que tiene un mapa de tipo “TileMap” que contiene escenas de tipo “Arbol”, “MinadeOro” y “MinadePiedra”, también tengo un nodo “Player” que se puede mover por el mapa. El mapa tiene una vista de tipo “top-down”, y cuando mi personaje se mueve detrás de un árbol, se ve delante. He visto que el inspector tiene un apartado “ordering” en casi todos los nodos.

Explícame paso a paso cómo implementar ordenación YSort en todos los elementos del mapa a través del inspector y la vista gráfica de los nodos.

Referencias: [Read me](#), [Estándares de Diseño](#), [Metodología de pruebas](#).